

ClientOnboard AI — Executive Report

Client-onboarding agent for a freelance web-development business · Week 2 Day 5 Capstone

Business goal

Freelancers and small web agencies lose time when new client requests arrive as unstructured free text with no consistent way to triage, price, or prioritize them. ClientOnboard AI converts a raw client request into a validated, categorized onboarding summary and a draft proposal, while keeping the final consequential action — sending that proposal to the client — under explicit human control. The goal is to remove first-pass manual triage work without removing human judgment from anything that commits the business or the client.

Architecture

The system receives a client request through a FastAPI endpoint, validates it against a strict schema (name, email, description length, budget and deadline bounds), applies a keyword-based safety guard, queries a local service catalog to recommend the best-fit offering, analyzes budget and deadline fit, drafts a client-facing proposal message, and then pauses at a human-approval checkpoint rather than sending anything automatically. A separate endpoint lets a human approve or reject the draft; only an approved run is considered ready for the next business step. Every run is logged with a unique run ID, the tool call it made, measured latency, and any errors, so the full path a request took is reconstructable after the fact.

- Nodes / steps: validate → safety guard → service lookup → analyze → draft proposal → human approval checkpoint → finalize
- Tool / external data source: local service catalog (data/services.json) — name, keywords, starting price, delivery time per service line
- State: the request, matched service, computed analysis, and draft proposal are threaded through each step and returned as one structured JSON object per run
- Human checkpoint: the proposal is never sent automatically — POST /approve/{run_id} is required before a run is treated as final

Framework choice rationale

This version is implemented as a raw deterministic pipeline — plain Python functions called in sequence — rather than LangGraph or CrewAI, and that is a deliberate v1 choice rather than a gap. The workflow is a single linear path with exactly one branch (the safety refusal) and one pause point (human approval); there is no cyclical reasoning, no self-correction loop, and no set of autonomous roles negotiating a shared goal, so a heavier orchestration framework would add abstraction without adding capability. Every step is a pure function with an exactly predictable output, which is what makes the evaluation harness fully reproducible run to run.

LangGraph becomes the right tool once the system needs what a raw pipeline can't express cleanly: branches driven by LLM judgment instead of fixed rules, a retry/self-correction loop around the drafting step, or the ability to persist and resume a paused run across a process restart — today, approvals are held in memory and lost if the server restarts. CrewAI was not selected because this is one job end-to-end, not several specialist roles collaborating; the analyzer, drafter, and safety check don't need separate agent identities or a delegation protocol between them. FastAPI provides a clean production-facing API boundary regardless of which orchestration layer sits behind it.

Evaluation results

Eight test cases were run, including two edge/adversarial cases (an invalid email and a credential-theft request), and scored programmatically from the agent's actual return value — not hand-typed — using evaluation_runner.py. All eight cases passed on the metrics below.

Case	Description	Status	Success	Accuracy	Tone	Safety	Latency (ms)
1	Normal bakery website request	pending_approval	1	1.0	1.0	1	0.62
2	Portfolio website request	pending_approval	1	1.0	1.0	1	0.10
3	Figma UI/UX prototype request	pending_approval	1	1.0	1.0	1	0.08
4	Bug fixing / maintenance request	pending_approval	1	1.0	1.0	1	0.07

5	EDGE: very short deadline (urgency)	pending_approval	1	1.0	1.0	1	0.06
6	EDGE: low budget vs. large app	pending_approval	1	1.0	1.0	1	0.07
7	EDGE: invalid email (bad input)	validation_error	1	1.0	N/A	1	N/A
8	ADVERSARIAL: credential-theft request	refused	1	1.0	N/A	1	N/A

Note on cost: no LLM is called in this version (classification is rule-based), so measured cost per run is \$0.00 by construction — this is accurate for the current system, not a placeholder, and is called out explicitly in Known Limitations below.

Most common failure pattern and fix

The eval table shows 100% pass on these eight cases, but that reflects what the automated criteria check for — it does not mean the system handles every request well. Manually reviewing the draft proposals surfaced a real, recurring weakness the pass/fail metrics don't catch: for short or underspecified requests (e.g., "fix bugs and update my website"), the recommendation is correct but the price/timeline estimate is presented with the same confidence as a fully-scoped request, which risks setting the wrong client expectation before scope is actually clarified. The concrete fix is to add a clarification step before the proposal is finalized: when the project description falls below a length/specificity threshold, the agent should return a short list of clarifying questions (pages/features needed, content readiness, integrations, revision expectations) instead of a firm estimate, and only produce the priced proposal once those are answered.

Known limitations

- Rule-based classification only — no LLM call, so recommendations are limited to the keyword coverage of the service catalog and cannot handle genuinely novel request types
- Local JSON catalog and in-memory approval store — not a database, and both are lost on process restart
- No authentication, persistent audit storage, distributed tracing, or retry/timeout policy around the (currently trivial) tool call
- Safety guard is a fixed keyword list, not a model-based classifier — it will miss rephrased or novel harmful requests
- Evaluation covers 8 cases; it is a smoke-test suite, not a statistically powered regression set

Recommended next steps

- Move the service catalog and approval records to a real database; add authentication and role-based access
- Replace rule-based classification with a production LLM behind structured-output validation, and add a self-correction/clarification loop for underspecified requests
- Migrate orchestration to LangGraph once the self-correction loop and resumable human-approval state are added — the current pipeline's steps map directly onto graph nodes
- Add retry/timeout policies around all tool and model calls, persistent audit logs, and real token-cost tracking once an LLM is introduced
- Build a larger regression dataset from real anonymized client requests and re-run it on every prompt/logic change; require human approval for any pricing, contract, or payment action, not just message-sending